Operating Systems Technologies

(Tecnologias de Sistemas Operativos)

Data Caching and Durability

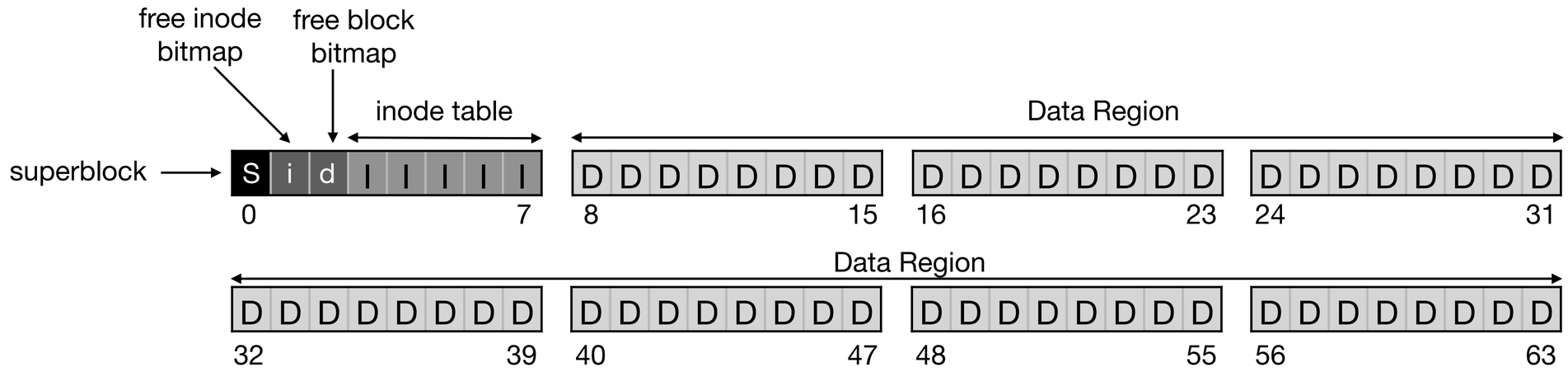
University of Minho

2025-2026

Remember the Recap!

Avoiding the disk I/O bottleneck

- Manipulating files requires several inode/data block operations!
- The OS heavily relies on in-memory caching
  - Avoids reading/writing to disk repeatedly
  - Disks are slower than memory

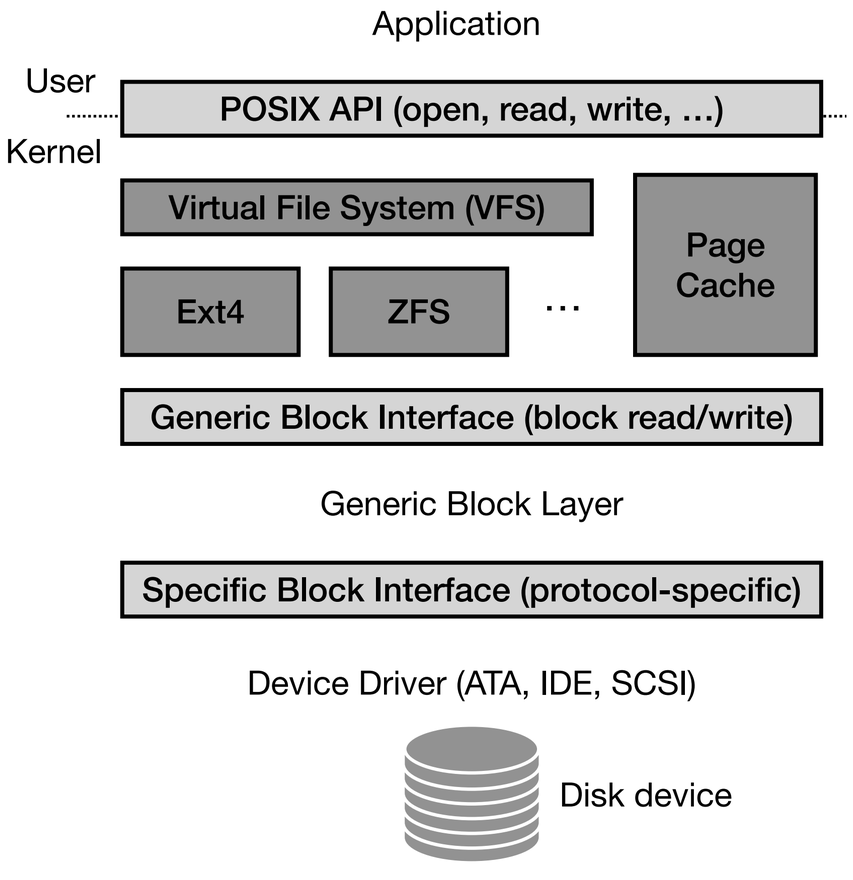


A really small disk with 64 blocks of 4KB

OS caches

File and block data, inodes, directories

- The **page cache** keeps popular content from
  - file data (*aka* file pages)
  - memory-mapped files
  - memory pages from processes (*aka* anonymous memory)
- An **inode cache** keeps popular inodes
- A **directory cache** (**dentry** cache) keeps popular content of directories (*i.e.,* name to inode pairs)
- The **buffer cache** keeps popular blocks from the generic block layer (*aka* block device)
  - Integrated with the page cache (since Linux Kernel 2.4)
- The interaction with OS caches is facilitated by the VFS



VFS example

Page Cache

Overview

- Entries are kept in a **page cache hash table**
  - Maps the file inode and offset to the page (content)
  - Serves both read (caching) and write (buffering) requests
- Entries are marked as **clean** if they were read but not updated, and marked as **dirty** if they were modified
- Dirty data is written to storage by background threads
  - Periodically, after a certain amount of time (*e.g.*, 5 seconds, 30 seconds), or when there are many dirty pages
  - File data is written to the corresponding file, and anonymous memory to the swap disk region

Page Cache

Replacement policy

- A modified form of **2Q** replacement chooses pages to evict when memory is full
  - Based on LRU, but more resilient to scan/uniform access patterns
    (*e.g.*, repeatedly scanning a large file - larger than available memory - will make LRU evict all other pages. Pages brought to memory will be evicted before being accessed again…)
- **Queue placement**
  - When accessed for the first time, a page is placed in the **inactive list** queue
  - When the page is accessed again, it is promoted to the **active list** queue
- **Replacement strategy**
  - Choose the candidate for eviction from the inactive list
  - Periodically move pages at the bottom of the active list (least recently used) to the inactive list
  - The active list should have about two-thirds of the total page cache size
- **Key idea:** When cyclic large-file accesses occur, these are contained in the inactive list

Optimizing I/O performance

Buffering

- The page cache, as well as other OS caches mentioned above, **buffer** updates to data and metadata!
  - It gives the opportunity to **batch** updates
  - Or even avoid I/O (e.g., when files are created and deleted in a short window)
  - **Problem:** If the system fails (e.g., due to a power loss), buffered data is lost!
  - **Buffering** data **improves performance** at the **cost of data durability** (a common tradeoff)

Crash Consistency

Challenges

- System failures are expected (e.g., sudden power losses). Ensuring a **crash-consistent** state of systems is a complex and hard problem!
- In the OS classes, we saw that this is a hard problem for file system developers
  - There are multiple crash scenarios where data and metadata may be **lost** or left **inconsistent**!
  - Led to **file system checkers**, **journaling**, and **Log-structured** designs
- Today, we are going to focus on applications that use the file system!

Crash Consistency

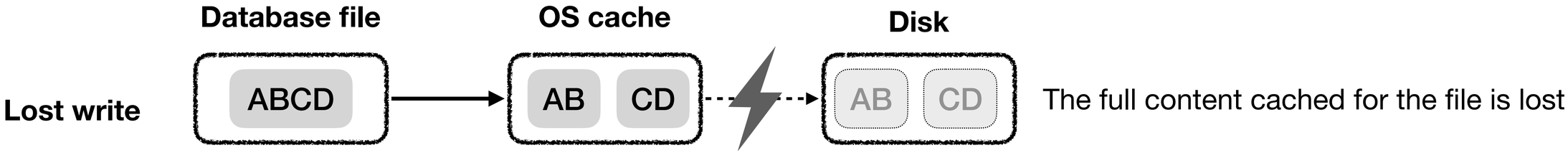
Data loss

- When a program calls *write(),* it asks the file system (FS) to persist data
  - However, the FS will **buffer** written data and metadata (page cache, inode cache, buffer cache, directory cache) in memory for a period of time (e.g., 5 seconds, 30 seconds…)
- If the machine crashes in the meantime, buffered data is lost…
  - Applications such as databases require strong persistence guarantees
  - But, what bad things can happen?

Crash Consistency

Data loss

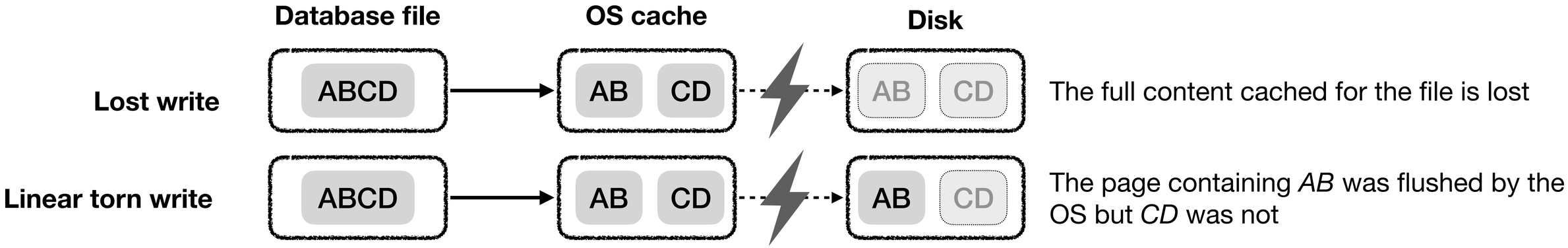
- When a program calls *write(),* it asks the file system (FS) to persist data
  - However, the FS will **buffer** written data and metadata (page cache, inode cache, buffer cache, directory cache) in memory for a period of time (e.g., 5 seconds, 30 seconds…)
- If the machine crashes in the meantime, buffered data is lost…
  - Applications such as databases require strong persistence guarantees
  - But, what bad things can happen?



Crash Consistency

Data loss

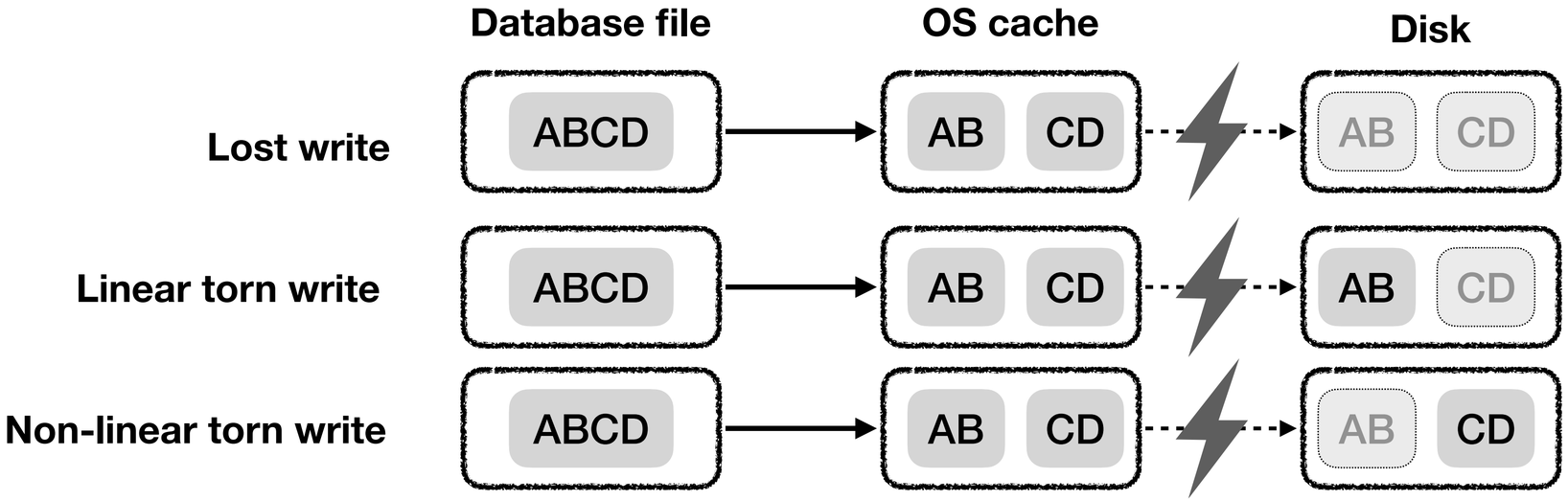
- When a program calls *write(),* it asks the file system (FS) to persist data
  - However, the FS will **buffer** written data and metadata (page cache, inode cache, buffer cache, directory cache) in memory for a period of time (e.g., 5 seconds, 30 seconds…)
- If the machine crashes in the meantime, buffered data is lost…
  - Applications such as databases require strong persistence guarantees
  - But, what bad things can happen?



Crash Consistency

Data loss

- When a program calls *write(),* it asks the file system (FS) to persist data
  - However, the FS will **buffer** written data and metadata (page cache, inode cache, buffer cache, directory cache) in memory for a period of time (e.g., 5 seconds, 30 seconds…)
- If the machine crashes in the meantime, buffered data is lost…
  - Applications such as databases require strong persistence guarantees
  - But, what bad things can happen?



The full content cached for the file is lost

The page containing *AB* was flushed by the OS but *CD* was not

The page containing *CD* was flushed, but *AB* was not (OS flushed pages out of order)

File System Interface

Flushing data and metadata to disk

- Can applications flush data from OS caches to disk?
- ***int fsync(int fd)*** - forces dirty data and metadata of a file descriptor to disk
  - In Linux, *fsync()* includes writing through or flushing the disk cache, while in OSx, the disk cache is not flushed explicitly
  - In some cases, one also needs to sync the directory where the file resides - often overlooked… (*i.e.*, if the file is newly created, the directory must know that it exists…)
- ***int fdatasync(int fd)*** - forces dirty data of a file descriptor to disk
  - does not flush modified metadata unless it is needed to correctly handle a subsequent data retrieval (*e.g.*, file's size)
- ***int sync()*** - causes all buffered modifications to files/directories metadata and data to be written to the underlying file systems
  - Costly operation performance-wise!

File System Interface

Flushing data and metadata to disk

- Upon file opening, one can also use special flags
- ***O_SYNC*** - By the time *write()* (or similar op) returns, file data and metadata have been transferred to disk, *i.e.*, as though each *write()* was followed by a call to *fsync()*
  - Costly performance-wise!
- ***O_DIRECT*** - minimizes cache effects of the I/O to and from the file
  - File I/O is done directly to/from the user-space buffers (takes advantage of DMA, if possible)
  - Useful when applications implement their own caching, and one wants to bypass OS caches
  - **Warning:** *O_DIRECT* attempts to transfer data synchronously, but does not provide the guarantees of *O_SYNC.* To guarantee synchronous I/O, both flags must be used

File System Interface

I/O performance vs data durability

- The family of *fsync()* operations is key to ensuring data durability, but the consequence is that I/O writes become significantly slower!
  - Try it out using simple C programs!
- Developers must be aware of this trade-off and design their applications accordingly, for instance
  - What information, in which files, needs to be persisted to ensure successful recovery?
  - When does this information need to be persisted?
  - Are there known techniques one can borrow from existing systems?
    - Write-ahead logging in databases
    - Journaling in file systems

More Information

- **Chapter 23** - Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. **Operating Systems: Three Easy Pieces.** Arpaci-Dusseau Books, 2018.
- **When Amnesia Strikes: Understanding and Reproducing Data Loss Bugs with Fault Injection** - Maria Ramos, João Azevedo, Kyle Kingsbury, José Pereira, Tânia Esteves, Ricardo Macedo, João Paulo, VLDB, 2024
- Want to know details about **crash consistency file system designs**?
  - **Chapters 42 and 43** - Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. **Operating Systems: Three Easy Pieces.** Arpaci-Dusseau Books, 2018.

Questions?

Extra Material

Crash Consistency

Challenges

- System failures are expected (e.g., sudden power losses). Ensuring a **crash-consistent** state of the file system under failures is a complex and hard problem!
- Multiple crash scenarios where data and metadata may be **lost** or left **inconsistent**!
  - The file's inode is updated, but the data block was never written (garbage pointer to data)
  - A data block is marked as used (bitmap), but the inode and data were not written (space leak)
  - Data and inode are updated, but the bitmap was not (inode/block marked as free and can be overwritten!)
  - Bitmap and data block are updated, but the inode was not (again, space leak)
  - And, unfortunately, there are other crash scenarios…
- Ideally, these 3 operations would be done **atomically**! This is difficult to do as the disk only commits one write at a time, and therefore, the **crash-consistency problem**

Crash Consistency

File system checker and journaling

- Use a **file system checker** (e.g., the fsck tool). Let inconsistencies happen, and then have a tool check for these and try to repair them

- **Journaling:** Use a write-ahead log[1] (journal) to write a transaction (containing pending data and metadata updates) before updating key structures.
  - Write (*i.e.*, checkpoint) data and metadata updates to their final locations later
  - If a crash occurs, the journal has the steps to try the request again (e.g., ext4)
  - Most FSs only register metadata updates in the journal.
    - Data is first written to disk. After confirming that it is persisted, metadata updates are added to the journal
    - No garbage pointers can occur, keeping metadata consistent while avoiding writing data updates to the journal (i.e., avoiding extra disk I/O requests)

[1] idea borrowed from the database community.

Crash Consistency

Log-structured file systems

- **Log-structured file systems (LFSs)** avoid overwriting (i.e., in-place updating) data on the disk. They always write to an unused portion of the disk and then later reclaim that old space through cleaning (e.g., ZFS)
- This approach is known as **copy-on-write** and enables highly efficient writing, as LFSs can buffer data and metadata updates in memory and then write them out together sequentially to disk
- We are leaving out several challenges from this very, very, very brief overview.
  - How can inodes be searched efficiently?
  - How are old data and metadata copies garbage collected?
  - How are consistent states recovered under failures?